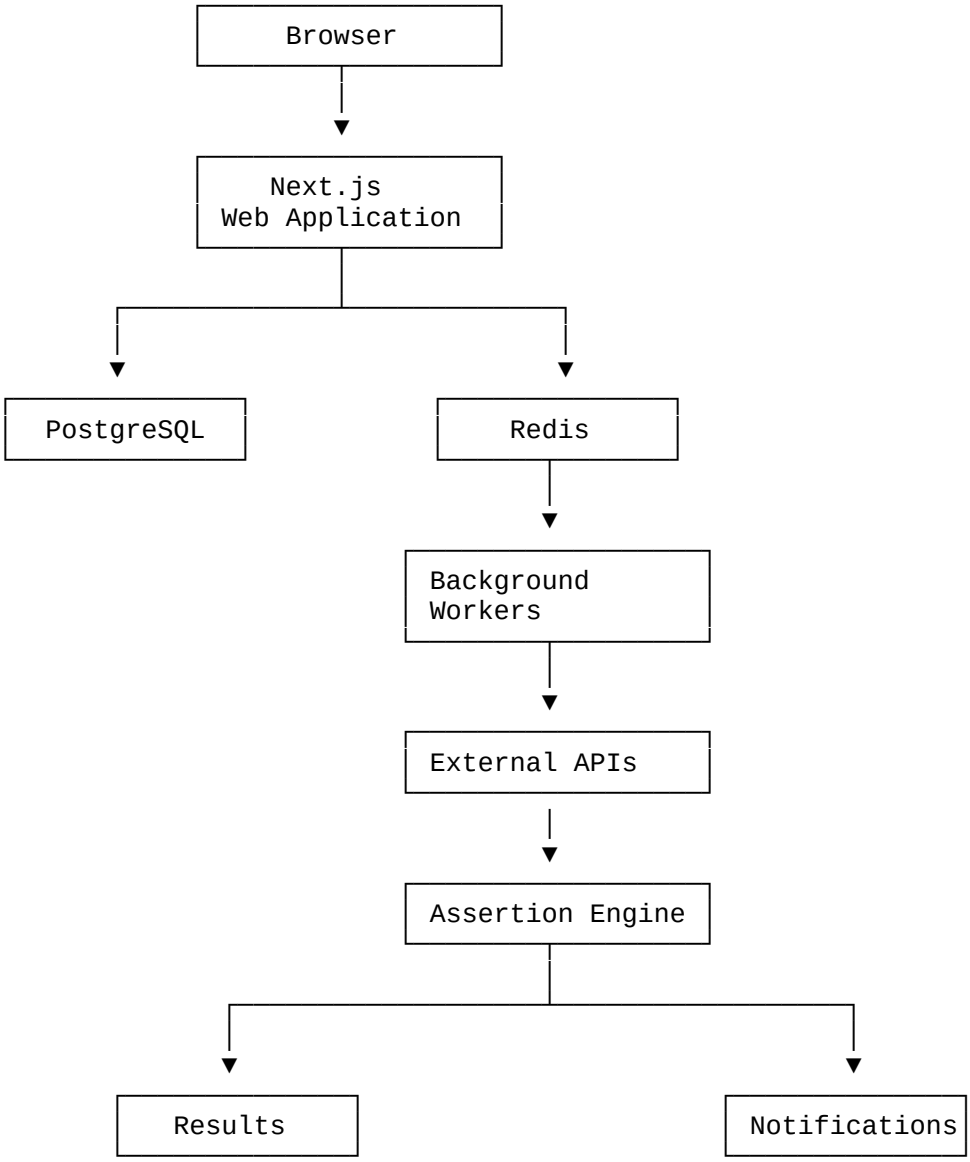
ARCHITECTURE.md - API Reliability & Testing Platform

1. Architecture Overview

The system will use a modular architecture:



---

2. Technology Stack

Frontend

```
Next.js
React
TypeScript
```

```
Tailwind CSS
```

Backend

Initially:

```
Next.js API routes / Route Handlers
```

As the application grows, the worker architecture remains separate.

---

3. Database

PostgreSQL

ORM:

Prisma

PostgreSQL stores:

- Users
- Projects
- Monitors
- Requests
- Assertions
- Test runs
- Test results
- Incidents
- Notifications
- Notification configurations

---

4. Queue

Redis + BullMQ

Redis handles:

- Job queues
- Scheduled jobs
- Retry state
- Worker coordination

BullMQ handles:

```
Monitor execution
Notification jobs
Incident processing
Cleanup jobs
```

---

5. Scheduler

The scheduler determines when monitors should execute.

Example:

```
Monitor:
GET /api/users

Schedule:
Every 5 minutes
```

Scheduler creates:

```
monitor-run job
```

The job enters Redis.

A worker retrieves it.

---

6. Worker

The worker is responsible for actual API testing.

Flow:

```
Receive Job
      ↓
Load Monitor
      ↓
Prepare Request
      ↓
Validate Target
      ↓
Execute HTTP Request
      ↓
Capture Response
      ↓
Run Assertions
      ↓
Determine Result
      ↓
Save Result
      ↓
Update Monitor Status
      ↓
Create/Update Incident
      ↓
Trigger Notification
```

---

7. Monitoring Engine

The monitoring engine should be independent from the web application.

This is important.

If a user dashboard crashes, monitoring should continue.

Likewise:

```
Next.js DOWN
```

should not mean:

```
Monitoring DOWN
```

---

8. Assertion Engine

The assertion engine receives:

```
Request
Response
Assertions
```

and returns:

```
PASS
```

or:

```
FAIL
```

Example:

```
{
  "assertion": "response_time",
  "expected": "< 500",
  "actual": 742,
  "passed": false
}
```

---

9. Test Result Lifecycle

```
Scheduled
    ↓
Queued
    ↓
Running
    ↓
Completed
    ↓
Passed / Failed
```

If execution fails unexpectedly:

```
Running
   ↓
Error
   ↓
Retry
   ↓
Failed
```

Retries must be carefully designed so temporary network failures don't immediately create false incidents.

---

10. Incident Engine

The incident engine determines when failures become incidents.

Example:

```
Test 1 → Failed
Test 2 → Failed
Test 3 → Failed
```

One incident:

```
INC-001
```

When the API recovers:

```
Test → Passed
```

The incident becomes:

```
Resolved
```

---

11. Notification System

Notification architecture:

```
Incident
   ↓
Notification Job
   ↓
Notification Worker
   ↓
Provider
```

MVP providers:

```
Email
Webhook
```

The notification system should be provider-based so new channels can be added later.

---

12. API Architecture

The application API should roughly follow:

```
/api/auth
/api/projects
/api/monitors
/api/monitors/:id
/api/monitors/:id/run
/api/monitors/:id/results
/api/monitors/:id/incidents
/api/incidents
/api/notifications
/api/settings
```

The exact endpoints should be defined later in `API.md`.

---

13. Data Flow

Creating a monitor

```
Browser
  ↓
Next.js
  ↓
Validate request
  ↓
PostgreSQL
  ↓
Monitor created
```

Executing a monitor

```
Scheduler
  ↓
Redis
  ↓
Worker
  ↓
Target API
  ↓
Assertion Engine
  ↓
PostgreSQL
  ↓
Incident Engine
  ↓
Notification Queue
  ↓
Email/Webhook
```

---

14. Failure Isolation

A monitored API could behave badly:

```
Infinite response
Huge response
Invalid TLS
Connection timeout
Malformed data
```

The worker must isolate each execution.

A problematic target must not be able to:

- Crash the worker
- Access internal infrastructure
- Consume unlimited memory
- Consume unlimited CPU
- Access secrets

---

15. SSRF Protection Architecture

Before a request is executed:

```
User URL
   ↓
Parse URL
   ↓
Resolve hostname
   ↓
Validate IP
   ↓
Block private/internal addresses
   ↓
Validate protocol
   ↓
Apply timeout
   ↓
Execute request
```

Only:

```
HTTP
HTTPS
```

should initially be supported.

---

16. Observability

The platform itself needs monitoring.

Track:

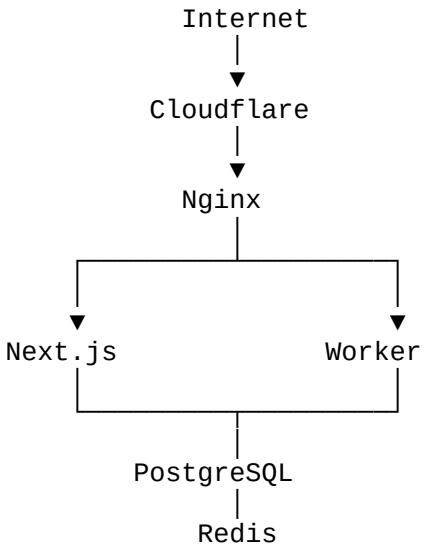
- Worker health
- Queue size
- Job failures
- Job duration
- Database health
- Redis health
- API response time
- Notification failures

We should avoid building an entire observability platform ourselves.

---

17. Deployment Architecture

Initial deployment:



Docker should be used for:

```
Next.js
Worker
PostgreSQL
Redis
```

For production, managed PostgreSQL/Redis can eventually replace self-hosted services if needed.

---

18. Repository Structure

I'd start with a monorepo:

```
api-reliability/
│
├── apps/
│   ├── web/
│   │   ├── app/
│   │   ├── components/
```

```
│   │   ├── lib/
│   │   └── ...
│   │
│   └── worker/
│       ├── jobs/
│       ├── services/
│       ├── engine/
│       └── ...
│
├── packages/
│   ├── database/
│   ├── types/
│   ├── validation/
│   └── config/
│
├── prisma/
│   ├── schema.prisma
│   └── migrations/
│
├── docker/
│
├── tests/
│
├── docker-compose.yml
├── package.json
├── README.md
├── PRD.md
└── ARCHITECTURE.md
```

---

19. Architectural Principle

The most important architectural decision is:

> **The web application should not be responsible for executing monitoring jobs.**

The web app manages:

Users + configuration + dashboards.

Workers manage:

Execution + testing + detection.

That separation gives us a much stronger foundation for scaling later.